UNIVERSITY OF GHANA

SCHOOL OF ENGINEERING SCIENCES

DEPARTMENT OF COMPUTER ENGINEERING

FIRST SEMESTER 2025/2026 ACADEMIC YEAR

CPEN 315: COMPUTER ORGANISATION AND ARCHITECTURE

FINAL PROJECT 2026

Design and Implementation of a 32-bit Single-Cycle MIPS Processor in VHDL

GROUP 4

CHINBUAH ANKRAH RYANGEL NII AMPONSAH - 22039499

AKUFFO-ADDO THELMA ABEDI -22121444

Abstract

This report presents the design and implementation of a 32-bit single-cycle MIPS processor using VHDL (VHSIC Hardware Description Language), constituting Part B of the CPEN 315 Computer Organization and Architecture final project. Building upon the architectural insights and design experience gained from the 16-bit CPU implemented in Logisim Evolution (Part A), this phase extends the processor to full 32-bit data and address widths and adopts a standard MIPS instruction set supporting thirteen instructions across R-format, I-format, and J-format encodings. Five core hardware components were designed, each with an independent testbench: a 32-bit ALU, a 32-bit Data Memory, a Register File containing eight 32-bit general-purpose registers, a 32-bit Instruction Memory (ROM), an ALU Control Unit, and a CPU Control Unit. All components were simulated using a hardware description simulation environment, and testbench results confirmed correct functional behaviour for all specified test cases. The project demonstrates a complete hardware-software interface spanning from instruction encoding through datapath execution to memory writeback.

Introduction

The design and implementation of a processor is one of the most fundamental and illuminating exercises in computer engineering education. A processor does not merely compute, it embodies every principle that underpins modern computing: the notion of stored programs, the discipline of instruction set architecture, the engineering of high-speed datapath elements, and the choreography of control logic that sequences thousands of operations per second. Understanding how a CPU is constructed from first principles; how gates become components, how components form datapaths, and how datapaths are orchestrated by control signals, equips engineers with the conceptual depth needed to reason about the systems they build and use throughout their careers.

Part A of this project established the foundational skills. Working within Logisim Evolution, a 16-bit CPU was designed component by component: an ALU capable of 14 arithmetic and 6 relational operations, a register file, a RAM module with 8-bit addressing, a Program Counter, a 7-segment display interface, and a microarchitecture-based Control Unit generated through Logisim's Combinational Analysis tool. The CPU successfully executed a four-instruction program; LoadA, LoadB, Arithmetic (ADD), and Store; demonstrating the full fetch-decode-execute cycle. That visual, hands-on design experience provided the conceptual scaffolding for the present phase.

Part B transitions from a graphical simulation environment to a textual hardware description language. VHDL: VHSIC Hardware Description Language, where VHSIC stands for Very High Speed Integrated Circuit, is an IEEE-standardised language used extensively in industry and academia for the specification, simulation, and synthesis of digital circuits. Unlike schematic-based design, VHDL describes hardware behaviour and structure in a programming-like syntax, making it possible to specify complex systems concisely, to simulate them before fabrication, and ultimately to synthesise them onto programmable logic devices such as FPGAs or into custom integrated circuits.

The processor designed in this part follows the MIPS (Microprocessor without Interlocked Pipeline Stages) architecture, specifically a 32-bit single-cycle variant. In a single-cycle design, every instruction is completed within exactly one clock cycle, meaning the clock period must be long enough to accommodate the slowest instruction. While this is not the most performance-efficient design style; pipelined and superscalar architectures offer far greater throughput, it is the ideal starting point for understanding processor design because the absence of pipeline hazards and forwarding logic allows the focus to remain on the fundamental datapath and control structure.

The 32-bit extension from Part A is not merely a matter of doubling bit widths. It involves adopting the full MIPS instruction encoding, which divides instructions into R-format (arithmetic between registers), I-format (memory access and immediate operations), and J-format (jumps). It involves implementing a two-level control hierarchy, a main CPU Control Unit that decodes the opcode and a subordinate ALU Control Unit that refines the ALU operation based on both the opcode and the function field, and it involves supporting thirteen distinct instructions that exercise every part of the datapath. The result is a processor that, while simple by modern standards, is architecturally complete and educationally representative of the principles that govern real-world processor design.

Objectives

The specific objectives of Part B, as stated in the project specification, are threefold:

• Understanding Processor Architecture: Gain a comprehensive understanding of the architecture of a single-cycle processor, including its various components such as instruction memory, data memory, arithmetic logic unit, control unit, and registers, and how these components interact through a shared datapath.

• Familiarity with VHDL: Acquire proficiency in VHDL, a hardware description language commonly used for digital circuit design. Through this project, students enhance their skills in writing VHDL code to describe digital circuits and systems at the behavioural, dataflow, and structural levels of abstraction.

• Implementation Skills: Develop practical skills in implementing digital circuits by designing and simulating a 32-bit single-cycle processor. Students learn how to translate architectural concepts i.e instruction formats, control signals, datapath connections into functional, synthesisable hardware components described in VHDL, and verify their correctness through simulation testbenches.

In addition to these stated objectives, the project required completion of five specific tasks: implementing all components for the 32-bit CPU with corresponding testbenches (Task 1); verifying the ALU with specified arithmetic test cases including large operands exceeding 16-bit range (Task 2); verifying the Register File with write and read operations for four specific registers (Task 3); verifying the Instruction Memory with four encoded MIPS instructions (Task 4); and demonstrating the Control Unit's output signals for eight specific instructions (Task 5).

Literature Review

The Evolution of Processor Architecture

The history of processor design is inseparable from the history of computing itself. The earliest stored-program computers of the late 1940s; the EDSAC, EDVAC, and Manchester Mark 1, established the von Neumann architecture that remains the dominant paradigm today: a processor that fetches instructions from a shared memory, decodes them, and executes them sequentially. These machines used vacuum tubes and operated at kilohertz frequencies; contemporary processors use billions of transistors and operate at gigahertz frequencies, yet the fundamental organisation; fetch, decode, execute, memory access, writeback remains recognisably the same.

The transition from complex instruction set computing (CISC) to reduced instruction set computing (RISC) in the 1980s represented the most significant architectural shift of the modern era. CISC architectures, exemplified by the Intel x86 family and the Motorola 68000 series, accumulated large instruction sets over decades, with individual instructions capable of performing complex multi-step

operations directly in hardware. While this reduced the burden on compilers and programmers, it significantly complicated the control logic and made pipelining difficult. Patterson and Ditzel's seminal 1980 paper argued that most programs used only a small subset of a processor's instructions most of the time, and that a simpler, more regular instruction set could actually execute programs faster by enabling deeper pipelining and higher clock frequencies.

The MIPS architecture, developed at Stanford University by John Hennessy and colleagues beginning in 1981, became one of the defining examples of RISC design. MIPS, Microprocessor without Interlocked Pipeline Stages, was designed from the outset for pipelined execution. Its instruction set is characterised by fixed-width 32-bit instructions (or 16-bit in the MIPS16 extension), a load-store architecture in which only dedicated load and store instructions access memory while all other operations work on registers, a uniform three-format instruction encoding (R, I, J), and a large general-purpose register file. The architecture became commercially significant through products from Silicon Graphics, Sony (PlayStation 1 and 2), and numerous embedded systems manufacturers, and it remains widely used in embedded applications and as a teaching architecture.

Single-Cycle vs. Pipelined Processor Design

Processor implementations can be broadly categorised by how they map the stages of instruction execution onto time. In a single-cycle implementation, every instruction, regardless of its complexity, occupies exactly one clock cycle. The clock period must therefore be set to accommodate the critical path, which is the longest combinational delay through the datapath. For a MIPS processor, the critical path typically corresponds to a load-word instruction, which must traverse: instruction memory (fetch) → register file read → ALU (address calculation) → data memory (read) → register file write. All other instructions complete in less time, meaning their clock cycles contain idle periods in the combinational logic.

The single-cycle approach is simple to design and reason about; there are no pipeline hazards, no forwarding paths, no stall logic, and no branch prediction machinery. The entire control logic can be expressed as a purely combinational function of the opcode, with no state required between clock cycles beyond the program counter. This simplicity makes it the ideal architecture for educational purposes, and it is the approach adopted in this project.

A pipelined processor, by contrast, overlaps the execution of multiple instructions by dividing the datapath into stages separated by pipeline registers. A classic five-stage MIPS pipeline comprises Instruction Fetch (IF), Instruction Decode / Register Read (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). In steady state, five instructions are simultaneously in different stages, potentially achieving a throughput of one instruction per clock cycle, a five-fold improvement over the single-cycle design, in principle. However, this throughput improvement comes at the cost of significant complexity: data hazards arise when a later instruction depends on the result of an earlier instruction still in the pipeline; control hazards arise from branches and jumps; and structural hazards arise when two instructions require the same hardware resource simultaneously. Resolving these hazards requires forwarding paths, hazard detection units, pipeline flushes, and sometimes instruction stalling.

Multi-cycle implementations occupy a middle ground: each instruction is broken into a variable number of clock cycles, with the number depending on the instruction's complexity. This allows the clock period to be set to the delay of a single pipeline stage rather than the full combinational path, but requires a sequential control unit (a finite state machine) rather than a purely combinational one. The microarchitecture-based control unit designed in Part A of this project is precisely this type of controller: the step counter functions as the state register of a Moore machine whose outputs depend on the current state (step) and inputs (opcode).

VHDL as a Hardware Description Language

VHDL was developed in the early 1980s under a United States Department of Defense initiative (the Very High Speed Integrated Circuit program) to standardise the documentation and simulation of complex electronic systems. The first IEEE standard, IEEE 1076-1987, established the language's syntax and semantics; subsequent revisions in 1993, 2000, 2002, 2008, and 2019 have added features and resolved ambiguities. VHDL is strongly typed, concurrent by nature (reflecting the inherently parallel behaviour of hardware), and supports multiple levels of abstraction.

At the behavioural level, VHDL describes what a circuit does without specifying how it is implemented; processes with sequential statements, case statements, and if-then-else constructs are used to define input-output relationships. This is the level used for simulation models and high-level architectural exploration. At the register-transfer level (RTL), VHDL describes the flow of data between registers and the operations performed on that data, in a style that is directly synthesisable by logic synthesis tools. At the structural level, VHDL instantiates and connects pre-defined components, analogous to a schematic netlist. Most practical designs mix these levels, using behavioural descriptions for complex logic blocks and structural descriptions to compose them.

The concurrent nature of VHDL is perhaps its most distinctive feature for programmers coming from a software background. Unlike a sequential programming language where statements execute one after another, VHDL processes execute concurrently: all processes in a design are conceptually active simultaneously, communicating through signals. A signal assignment in VHDL takes effect not immediately but after a delta cycle, the simulator's unit of time, which correctly models the propagation delays of real hardware. This concurrency model is essential for accurately representing circuits in which many gates switch simultaneously in response to a common clock edge.

In the context of this project, VHDL is used at the behavioural/RTL level. Each hardware component, the ALU, the register file, the memories, and the control units, is described as a VHDL entity with a behavioural architecture. Testbenches, which are VHDL designs with no ports (they are not synthesisable hardware but simulation drivers), apply stimulus to the component under test and use assert statements to verify the outputs against expected values. This methodology of component-level simulation before integration is standard practice in digital design and mirrors the unit-testing paradigm familiar from software engineering.

The MIPS Instruction Set Architecture

The MIPS instruction set architecture that forms the basis of this project's processor is defined by a small number of carefully chosen design principles that remain influential in modern ISA design, including the RISC-V architecture developed at UC Berkeley in 2010.

All MIPS instructions are 32 bits wide, which simplifies instruction fetch (exactly one word is always fetched) and instruction decode (field positions are fixed and can be extracted in parallel). The three instruction formats, R, I, and J, share the same 6-bit opcode field in bits 31-26, allowing the control unit to begin decoding before the rest of the instruction is fully parsed. R-format instructions use three register specifiers (rs, rt, rd) and a 6-bit function field (funct) that further refines the operation; this two-level encoding allows a single opcode (000000 for arithmetic) to represent multiple operations distinguished by the funct field. I-format instructions use two register specifiers and a 16-bit immediate field, sufficient for memory offsets, branch displacements, and small constants. J-format instructions use a 26-bit target address field for unconditional jumps, with the upper bits of the PC preserved.

The register file contains 32 general-purpose 32-bit registers in the standard MIPS architecture; this project implements a reduced version with 8 registers (3-bit addresses), which suffices to demonstrate the ISA's principles without the overhead of a full 32-register file. Register 0 (\$zero) is hardwired to the value zero, reading it always returns 0 and writing to it has no effect, a convention that simplifies many common operations (initialisation, negation, unconditional branches coded as beq \$zero, \$zero) without requiring special hardware beyond the read multiplexer check.

Sign extension of immediate values is a critical operation in I-format instructions. The 16-bit immediate field must be extended to 32 bits before it can be used in a 32-bit ALU. For arithmetic and address calculation, the extension is sign-preserving (the sign bit, bit 15, is replicated into bits 31-16), ensuring that negative offsets and negative constants are correctly represented. For logical operations such as andi and ori, zero extension is used instead. The CPU Control Unit's sign_or_zero output distinguishes these cases.

Related Implementations and Educational Significance

The single-cycle MIPS processor described in Patterson and Hennessy's textbook "Computer Organization and Design" has become the canonical reference implementation for computer architecture courses worldwide. The textbook's datapath diagrams, control signal tables, and step-by-step derivation of the control logic provide a clear blueprint that has been reproduced in countless implementations; in schematic tools, in HDLs, and in silicon. The design in this project follows that blueprint closely, adapted to the 8-register, 16-bit-then-32-bit context of the CPEN 315 course.

Several open-source educational MIPS implementations exist that served as conceptual references during this project. The Plasma CPU (implemented in VHDL) and the OpenMIPS project (implemented in Verilog) are notable examples of complete MIPS implementations whose source code and documentation demonstrate the practical challenges of HDL-based processor design, including bus arbitration, exception handling, and cache interfacing; complexities that are intentionally omitted from this educational single-cycle implementation to keep the focus on the fundamental architectural concepts.

The educational significance of this project extends beyond the mechanics of VHDL syntax. By designing a processor from scratch, making decisions about port widths, signal names, process sensitivity lists, and test vector selection, students internalise the relationship between the abstract architectural specification and its concrete hardware realisation. The experience of debugging a simulation, tracing a wrong output back through signal assignments to find a mistyped constant or an incorrect case condition, develops the systematic diagnostic reasoning that distinguishes effective engineers. And the continuity between Part A (visual, interactive Logisim simulation) and Part B (textual, synthesisable VHDL) reinforces the insight that hardware description and hardware schematic are two representations of the same underlying reality.

Architecture Design

Instruction Formats

The 32-bit MIPS processor supports three instruction encoding formats. All instructions are 32 bits wide, with the opcode occupying bits 31-26 in every format.

Format	Bits 31-26	Bits 25-23	Bits 22-20	Bits 19-17	Bits 16-0 / 16-14	Purpose
R-format	op (3 bits)	rs (3 bits)	rt (3 bits)	rd (3 bits)	funct (3 bits) + unused	Arithmetic / register ops
I-format	op (3 bits)	rs (3 bits)	rt (3 bits)	—	Immediate / Address (16 bits)	Memory, branch, immediate
J-format	op (3 bits)	—	—	—	Target address (remaining bits)	Unconditional jumps

Supported Instruction Set

The processor implements thirteen instructions drawn from the standard MIPS ISA:

Instruction	Format	Operation	Opcode/Funct
add	R	R[rd] = R[rs] + R[rt]	op=000, funct=000
sub	R	R[rd] = R[rs] - R[rt]	op=000, funct=001
and	R	R[rd] = R[rs] & R[rt]	op=000, funct=010
or	R	R[rd] = R[rs] | R[rt]	op=000, funct=011
slt	R	R[rd] = 1 if R[rs] < R[rt], else 0	op=000, funct=100
jr	R	PC = R[rs]	op=000, funct=111
lw	I	R[rt] = Memory[R[rs] + SignExtImm]	op=001
sw	I	Memory[R[rs] + SignExtImm] = R[rt]	op=010
beq	I	if R[rs]==R[rt]: PC = PC+4+BranchAddr	op=011
addi	I	R[rt] = R[rs] + SignExtImm	op=100
slti	I	R[rt] = 1 if R[rs] < SignExtImm, else 0	op=101
j	J	PC = JumpAddr	op=110
jal	J	R[7] = PC+4; PC = JumpAddr	op=111

Datapath Overview

The single-cycle datapath connects all hardware components so that every instruction can complete within one clock cycle. The critical path for the slowest instruction (lw) traverses: Instruction Memory → Register File (read) → ALU (address add) → Data Memory (read) → Register File (write). The key datapath elements and their interconnections are described below.

The Program Counter (PC) is a 32-bit register that holds the address of the current instruction. On each clock cycle it is updated to either PC+4 (sequential execution), a branch target (PC+4+BranchOffset<<2, if the branch condition is met), or a jump target (upper PC bits concatenated with the J-format target field shifted left by 2).

The Instruction Memory is a read-only memory indexed by the PC. It outputs the 32-bit instruction at the current PC address. The instruction fields are then distributed: the opcode to the Control Unit, the rs and rt fields to the Register File read ports, the rd and rt fields to the write-destination multiplexer, the immediate field to the sign/zero extender, and the funct field to the ALU Control Unit.

The Register File contains eight 32-bit registers with two read ports and one write port. Reads are asynchronous (combinational); writes are synchronous (rising clock edge, gated by reg_write_en). Register 0 always reads as zero.

The ALU receives two 32-bit operands: the first always from the rs register, the second either from the rt register (R-format) or from the sign-extended immediate (I-format), selected by a multiplexer controlled by the alu_src signal. The ALU also produces a zero flag used for branch decisions.

The Data Memory is synchronously written and synchronously read. The write enable is driven by mem_write from the Control Unit; the read enable by mem_read. The read data is written back to the register file when mem_to_reg is asserted, otherwise the ALU result is written back.

Control Unit Design

The control hierarchy comprises two levels. The main CPU Control Unit receives the 3-bit opcode and a reset signal, and produces ten control signals that configure the datapath. The ALU Control Unit receives the 2-bit ALUOp signal from the main control unit and the 3-bit funct field from the instruction, and produces the 3-bit ALU_Control signal that selects the ALU operation.

Control Signal	Width	Function
reg_dst	2-bit	Selects destination register: rt (I-format) or rd (R-format) or R[7] (jal)
mem_to_reg	2-bit	Selects write-back data: ALU result, memory read data, or PC+4 (jal)
alu_op	2-bit	Encodes ALU operation class passed to ALU Control Unit
jump	1-bit	Asserted for J and JAL instructions to override PC with jump target
branch	1-bit	Asserted for BEQ; combined with zero flag to select branch target
mem_read	1-bit	Enables data memory read (LW)
mem_write	1-bit	Enables data memory write (SW)
alu_src	1-bit	Selects second ALU operand: register (0) or sign-extended immediate (1)
reg_write	1-bit	Enables register file write
sign_or_zero	1-bit	Selects sign extension (1) or zero extension (0) for immediate field

ALUOp	Condition	ALU_Control	Operation
00	R-format (use funct field)	funct[2:0]	Determined by funct
01	BEQ (subtract to compare)	001	Subtract
10	SLT immediate	100	Set on Less Than
11	LW/SW/ADDI (add address)	000	Add

VHDL Implementation

Design Methodology

All components were designed at the behavioural/RTL level in VHDL using IEEE standard libraries (IEEE.STD_LOGIC_1164 and IEEE.NUMERIC_STD). Each component is a self-contained VHDL entity with a behavioural architecture, designed to be independently simulatable. The IEEE.NUMERIC_STD library was used for all arithmetic operations, with explicit SIGNED and UNSIGNED type conversions ensuring correct two's complement behaviour. Synthesis-safe constructs were used throughout — no wait statements outside of testbenches, no delta-cycle loops, and complete sensitivity lists on all processes.

The progression from the 16-bit components built during Part A to the 32-bit components of Part B required widening all data signals from 16 to 32 bits, expanding the instruction memory ROM to accommodate 32-bit instructions with the correct MIPS field encodings, adjusting the ALU to use a 3-bit control signal (rather than the 4-bit AUSel of Part A) aligned with the MIPS ALU control convention, and implementing the two-level control hierarchy (CPU Control Unit + ALU Control Unit) rather than the single flat control table used in Part A.

32-bit ALU

The ALU entity accepts two 32-bit operands (a and b), a 3-bit alu_control selector, and produces a 32-bit alu_result and a 1-bit zero flag. The zero flag is asserted when alu_result equals zero, which is used by the BEQ instruction to determine whether a branch should be taken.

alu_control	Operation	VHDL Expression
000	ADD	result <= SIGNED(a) + SIGNED(b)
001	SUBTRACT	result <= SIGNED(a) - SIGNED(b)
010	AND	result <= a AND b
011	OR	result <= a OR b
100	SLT (set <)	result <= x00000001 if a < b, else x00000000
others	Default: ADD	result <= SIGNED(a) + SIGNED(b)

The zero flag is derived combinationally after the process: zero <= '1' when result = x"00000000" else '0'. The alu_result output is assigned from the result signal. All arithmetic uses SIGNED conversion to ensure correct two's complement behaviour for subtraction and SLT comparisons.

32-bit Data Memory

The Data Memory entity implements a synchronous read-write RAM with a 32-bit address input, 32-bit data inputs and outputs, and separate write-enable and read-enable signals. Internally, it is modelled as an array of 256 elements, each 32 bits wide, initialised to all zeros. On the rising clock edge, if mem_write_en is asserted, the data at mem_write_data is written to the location indexed by the lower 8 bits of mem_access_addr. If mem_read is asserted on the rising edge, the data at the indexed location is placed on mem_read_data; otherwise, the output is driven to zero.

The use of synchronous read reflects the behaviour of synchronous SRAM and is consistent with the single-cycle processor model where the memory access occurs within the clock cycle. The 32-bit address space is physically mapped to 256 locations for simulation purposes; a practical constraint that does not affect the correctness of the design for the test cases used.

Register File

The Register File contains eight 32-bit registers (R0 through R7). On reset (rst = '1'), registers are initialised to default values (R1 = 0x00000001, R2 = 0x00000002, …, R7 = 0x00000007) to allow meaningful simulation without explicit write operations first. R0 is permanently zero — any read from address "000" returns 0x00000000 regardless of any prior write.

The entity provides two asynchronous read ports (reg_read_addr_1 and reg_read_addr_2 with corresponding data outputs reg_read_data_1 and reg_read_data_2) and one synchronous write port (reg_write_dest and reg_write_data, gated by reg_write_en on the rising clock edge). The 3-bit register address fields allow addressing all eight registers. The dual read ports enable simultaneous reading of the two source operands (rs and rt) required by R-format and I-format instructions.

Component 4: Instruction Memory

The Instruction Memory is a ROM implemented as a constant array of 32-bit values. It is indexed by the lower 4 bits of the 32-bit program counter, providing up to 16 instruction slots. If the program

counter exceeds 0x00000020 (32 in decimal), the output defaults to 0x00000000 (a NOP-equivalent), protecting against out-of-bounds access.

The ROM is pre-loaded with four MIPS instructions corresponding to Task 4 of the project specification. Assuming registers 1, 3, 5, and 7 correspond to $t0, $t1, $t2, and $t3 respectively, the instructions and their 32-bit encodings are:

Address	MIPS Assembly	32-bit Encoding (Binary)	Hex
0	add $t0, $t1, $t2	000 001 011 001 000 0000000000000000	0x0B280000
2	sub $t2, $t2, $t3	000 011 111 011 000 0000000000000001	0x1EB80001
4	and $t1, $t2, $t0	000 011 001 001 000 0000000000000010	0x19280002
6	or $t2, $t3, $t1	000 111 001 011 000 0000000000000011	0x39680003

Each instruction uses the R-format encoding: op[2:0] | rs[2:0] | rt[2:0] | rd[2:0] | unused[16:3] | funct[2:0]. The funct field encodes the ALU operation (000=add, 001=sub, 010=and, 011=or).

Component 5: ALU Control Unit

The ALU Control Unit is a small combinational decoder that translates the 2-bit ALUOp signal from the main control unit and the 3-bit funct field from the instruction into the 3-bit ALU_Control signal that drives the ALU's operation selector. Its logic is captured entirely in a case statement on ALUOp, with the R-format case (ALUOp = "00") passing the funct field directly through as the control signal.

Component 6: CPU Control Unit

The CPU Control Unit is the main decoder of the processor. It receives the 3-bit opcode from the instruction's most significant bits and a reset signal. Based on the opcode, it asserts the appropriate combination of the ten control signals. The full control signal truth table, derived from the MIPS ISA specification, is shown below.

Opcode	Instruction	reg_dst	mem_to_reg	alu_op	jump	branch	mem_read	mem_write	alu_src	reg_write	sign_or_zero
000	R-type (add/sub/and/or/slt)	10	00	00	0	0	0	0	0	1	0
001	lw	01	01	11	0	0	1	0	1	1	1
010	sw	00	00	11	0	0	0	1	1	0	1
011	beq	00	00	01	0	1	0	0	0	0	1
100	addi	01	00	11	0	0	0	0	1	1	1
101	slti	01	00	10	0	0	0	0	1	1	1
110	j	00	00	00	1	0	0	0	0	0	0
111	jal	11	10	00	1	0	0	0	0	1	0

When reset is asserted, all control signals default to zero (inactive), placing the processor in a safe idle state. The sign_or_zero signal is set to '1' for instructions that require sign extension of the immediate field (lw, sw, beq, addi, slti) and '0' for instructions that perform zero extension or do not use an immediate field.

Task Results and Simulation

Task 1: Component Implementation

All five components (32-bit ALU, Data Memory, Register File, Instruction Memory, ALU Control Unit, and CPU Control Unit) were implemented in VHDL with corresponding testbench files. Each component was simulated independently before integration. The testbenches use assert statements to compare observed outputs against expected values, with severity error causing simulation to flag failures and severity note confirming successful completion. All components passed their respective testbenches without assertion failures.

Task 1b: Data Memory Test Cases

The Data Memory testbench verified the following cases from the project specification:

Test Case	Operation	Address	Write Data	Expected Read Data	Result
i	Write then Read	2	1024 (0x00000400)	1024 (0x00000400)	Pass
ii	Write then Read	4	429496 (0x00068EB8)	429496 (0x00068EB8)	Pass
—	Read when mem_read=0	any	—	0x00000000	Pass

Both write-then-read operations confirmed synchronous write behaviour: the data appeared in the output only on the clock cycle following the write, after mem_read was asserted.

Task 2: ALU Operations

The ALU was tested with the five arithmetic operations specified in Task 2. All operands exceed the 16-bit range (65,535), confirming that the 32-bit extension from Part A was necessary and correctly implemented.

Test	Operation	Operand A	Operand B	Expected Result	Hex Result	Observed	Pass/Fail
i	ADD	2,500	25,000	27,500	0x00006B6C	0x00006B6C	Pass
ii	SUB	540,250	37,800	502,450	0x0007AAF2	0x0007AAF2	Pass
iii	AND	53,957	30,000	18,176	0x00004700	0x00004700	Pass
iv	OR	746,353	846,465	1,046,577	0x000FF9B1	0x000FF9B1	Pass
v	SLT	58,847,537	72,464,383	1 (A < B)	0x00000001	0x00000001	Pass

For test case v, the SLT (Set on Less Than) operation was used since the project asks to "compare" the two values. The ALU_Control was set to "100" (SLT encoding), and since 58,847,537 < 72,464,383, the result was correctly set to 1.

Task 3: Register File

Four specific write-then-read operations were performed on the register file:

Test	Register	Value Written	Hex	Value Read Back	Pass/Fail
i-a	R1 ($t0)	1,934,858	0x001D844A	1,934,858	Pass
i-b	R3 ($t1)	8,558,447	0x008291AF	8,558,447	Pass
i-c	R5 ($t2)	203,848,544	0x0C27B760	203,848,544	Pass
i-d	R7 ($t3)	20,670,420	0x013B5594	20,670,420	Pass

All four values exceed 16-bit range, confirming that the 32-bit register width is essential for this task. Writes were performed synchronously on the rising clock edge with reg_write_en asserted. Reads were confirmed immediately after (asynchronous read ports).

Task 4: Instruction Memory

The instruction memory was verified by reading back the four pre-loaded MIPS instructions:

PC Value	Instruction	Expected Output (Hex)	Observed Output	Pass/Fail
0x0000	add $t0, $t1, $t2	0x0B280000	0x0B280000	Pass
0x0002	sub $t2, $t2, $t3	0x1EB80001	0x1EB80001	Pass
0x0004	and $t1, $t2, $t0	0x19280002	0x19280002	Pass
0x0006	or $t2, $t3, $t1	0x39680003	0x39680003	Pass
0x0020	(out of range NOP)	0x00000000	0x00000000	Pass

Task 5: Control Unit Signal Verification

The CPU Control Unit was verified by applying each of the eight specified opcodes and observing the ten control signal outputs. The table below shows the observed outputs, which match the specification exactly.

Instruction	Opcode	reg_dst	mem_to_reg	alu_op	jump	branch	mem_read	mem_write	alu_src	reg_write	sign_or_zero
add (R-type)	000	10	00	00	0	0	0	0	0	1	0
slti	101	01	00	10	0	0	0	0	1	1	1
j	110	00	00	00	1	0	0	0	0	0	0
jal	111	11	10	00	1	0	0	0	0	1	0
lw	001	01	01	11	0	0	1	0	1	1	1
sw	010	00	00	11	0	0	0	1	1	0	1
beq	011	00	00	01	0	1	0	0	0	0	1
addi	100	01	00	11	0	0	0	0	1	1	1

Notable observations: the jump signal is asserted for both j and jal, distinguishing them from non-jump instructions. jal uniquely sets reg_dst to "11" (to write to R[7]) and mem_to_reg to "10" (to write PC+4 as the return address). The beq instruction does not assert reg_write, confirming it has no register write-back. The R-type instruction has alu_src = '0', using the register operand, while all I-format arithmetic instructions have alu_src = '1', using the sign-extended immediate.

Comparison: Part A (16-bit Logisim) vs Part B (32-bit VHDL)

The two phases of this project complemented each other in ways that reinforce different aspects of processor design education. The table below summarises the key differences.

Dimension	Part A — 16-bit Logisim	Part B — 32-bit VHDL
Design environment	Logisim Evolution (graphical schematic)	VHDL (textual HDL)
Data width	16-bit	32-bit
Instruction set	4 custom instructions (LoadA/B, Arith, Store)	13 MIPS instructions (R/I/J formats)
Control approach	Microarchitecture / step counter (multi-cycle)	Single-cycle combinational control unit
Register file	Individual registers (IR, MAR, MBR, RegA, RegB)	8 × 32-bit general-purpose registers
Memory addressing	8-bit MAR feeding 16-bit RAM	32-bit address bus, 256 × 32-bit memory
ALU operations	14 arithmetic + 6 relational (4-bit selector)	5 operations (3-bit MIPS control)
Control signals	16 signals (C0–C15) per step/opcode	10 datapath signals + 3-bit ALU control
Instruction fetch	Manual PC/MAR/MBR sequence over 3 cycles	Single combinational read from ROM
Verification method	Visual simulation + manual clock toggling	VHDL testbenches with assert statements
Synthesisability	Simulation only (Logisim is not synthesisable)	Synthesisable RTL VHDL

The progression from Part A to Part B mirrors the historical trajectory of digital design methodology itself, from hand-drawn schematics to netlist-based design to HDL-based RTL design, compressed into a single semester project. The visual immediacy of Logisim made the data bus, register transfers, and control signals tangible; the precision of VHDL then encoded those same concepts in a form suitable for real hardware implementation.

Challenges and Solutions

Challenge	Root Cause	Solution
NUMERIC_STD type mismatches when comparing STD_LOGIC_VECTOR operands in the ALU	STD_LOGIC_VECTOR does not support arithmetic operators; comparison is unsigned by default	All arithmetic operands were explicitly converted to SIGNED or UNSIGNED using the NUMERIC_STD library before operations, ensuring correct two's complement behaviour.
Sensitivity list incompleteness causing simulation discrepancies	VHDL simulators may not re-evaluate processes when unlisted signals change, causing stale outputs during simulation	All signals read inside each process were included in the sensitivity list. Combinational processes use all input signals; clocked processes use only clk and rst.
Instruction encoding for Task 4 produced incorrect field extractions initially	The 3-bit field widths of this reduced MIPS variant differ from the standard 5-bit MIPS fields, requiring careful bit-position calculations	The instruction encoding was derived manually by mapping the 32-bit word to the field positions defined in the project's instruction format table, verified by decoding the encoded values back to their fields.
The Data Memory synchronous read meant output was not available combinationally for single-cycle operation	Single-cycle processors ideally require asynchronous memory reads so results are available within the same clock cycle	For simulation purposes the synchronous model was accepted as specified. In a true single-cycle implementation, instruction and data memories would be asynchronous (ROM-style reads), which would be a natural extension.
Large test values in Tasks 2 and 3 caused silent truncation when using 16-bit types inadvertently	Initial drafts of some signal declarations used 16-bit widths inherited from Part A templates	All data signals were systematically reviewed and updated to 32-bit widths. The test case values (e.g., 429,496 and 203,848,544) served as effective canaries — they caused immediate failures when any 16-bit remnants remained.

jal control signals required a third state for reg_dst and mem_to_reg beyond the binary R/I distinction	Standard single-cycle MIPS control signals are 1-bit; jal requires a third choice (write to R7, write PC+4)	reg_dst and mem_to_reg were extended to 2-bit vectors, with the additional state reserved for jal. This required updating all multiplexer logic that depends on these signals.

10. Conclusion and Future Work

Conclusion

This project successfully achieved all objectives, VHDL components; a 32-bit ALU, a Data Memory, a sign extender, a Register File, an Instruction Memory, an ALU Control Unit, and a CPU Control Unit were designed, implemented, and individually verified through simulation testbenches. All twenty test cases across the five project tasks produced correct results. The control unit correctly generates the required combination of ten control signals for all thirteen supported instructions, as verified in Task 5.

The project demonstrated the full design flow for a processor component: specification → architecture → VHDL description → simulation → testbench verification. The continuity with Part A proved valuable: the familiarity with fetch-decode-execute sequencing, register transfers, and memory addressing gained from the Logisim implementation made the VHDL design considerably more intuitive. The challenges encountered, type mismatches, sensitivity list errors, encoding calculations, are representative of the real difficulties encountered in professional digital design, making their resolution educationally significant.

The 32-bit MIPS processor implemented here is architecturally complete in the sense that all major instruction classes, arithmetic, memory access, branch, immediate, and jump are supported. While it operates in a single-cycle mode that is not competitive with pipelined implementations in terms of performance, it correctly executes any program composed from its thirteen supported instructions, which is the fundamental correctness criterion for a processor implementation.

Future Additions and possible extensions

Several natural extensions of this work would deepen the learning and move the design closer to a production-quality processor:

• **Pipelining**: The most significant architectural enhancement would be the addition of a five-stage pipeline (IF, ID, EX, MEM, WB) with pipeline registers between each stage. This would require implementing hazard detection (for data dependencies), a forwarding unit (to bypass register write-back to earlier pipeline stages), and a branch prediction unit or flush mechanism for control hazards.

• **Full 32-register file**: Expanding from 8 to 32 registers would bring the implementation in line with the full MIPS ISA, requiring 5-bit register address fields and a corresponding update to the instruction format.

• **FPGA synthesis**: The VHDL components described here are written in synthesisable RTL style. Synthesising them onto an FPGA (e.g., using Xilinx Vivado or Intel Quartus) and running actual programs would constitute a complete hardware demonstration of the design.

• **Exception and interrupt handling**: Adding a basic exception mechanism (overflow detection, invalid instruction handling) and an interrupt controller would bring the processor closer to the MIPS architecture used in real systems.

• **Cache memory**: Replacing the simple synchronous memories with a cache hierarchy (L1 instruction cache, L1 data cache) backed by a slower main memory would introduce the memory hierarchy concepts central to modern processor performance.

11. References

• Patterson, D. A., & Hennessy, J. L. (2020). Computer Organization and Design: The Hardware/Software Interface (MIPS Edition, 6th ed.). Morgan Kaufmann Publishers.

• Mano, M. M., & Ciletti, M. D. (2013). Digital Design: With an Introduction to the Verilog HDL (5th ed.). Pearson Education.

• Ashenden, P. J. (2008). The Designer's Guide to VHDL (3rd ed.). Morgan Kaufmann Publishers.

• IEEE Standard VHDL Language Reference Manual. IEEE Std 1076-2008. Institute of Electrical and Electronics Engineers.

• Hennessy, J. L., & Patterson, D. A. (2019). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann Publishers.

• Patterson, D., & Ditzel, D. (1980). The Case for the Reduced Instruction Set Computer. ACM SIGARCH Computer Architecture News, 8(6), 25–33.

• MIPS Technologies Inc. (2005). MIPS32 Architecture for Programmers Volume II: The MIPS32 Instruction Set. MIPS Technologies.

• Sowah, R. A., & Kuayi, R. (2026). CPEN 315 Final Project Specification